



AURA / RIZAL

Flutter App Reviewer

A beginner-friendly study guide for your interview

This reviewer explains the Aura (codename RIZAL) mobile app from the ground up: what it does, the technology behind it, and how the code is organized. It moves from absolute basics, through core Flutter concepts, to the harder architecture topics interviewers love to ask about. Every piece of jargon is explained in plain language and collected in a glossary near the end.

Snapshot

What	A mobile app for school attendance + student governance
Platforms	Android & iOS (one shared codebase) + web preview
Built with	Flutter (UI toolkit) + Dart (language), Material 3
Talks to	An existing cloud backend (FastAPI) over the internet
Package	aura_app - appld.com.aura.aura_app
Status	Pre-1.0, ~v1.36, building toward 4-workspace parity

Generated from the live codebase - scope: frontend Flutter app only.

What's inside

0	The 30-second pitch	Explain the app in one breath
1	Absolute basics	App, frontend/backend, client-server, API, JSON
2	Flutter & Dart fundamentals	Widgets, state, build, hot reload, packages
3	How this app is structured	Folders, layers, the four workspaces
4	State management (Riverpod)	Providers, Notifiers, watch vs read
5	Navigation (go_router)	Routes and the login/redirect gate
6	Talking to the backend	Dio, REST, JWT tokens, JSON models, paging
7	Key features	Face scan, geofencing, governance, AI, reports
8	Design & theming	Tokens, dark mode, branding, motion, glass nav
9	Security	Tokens, secure storage, HTTPS, anti-spoofing
10	Quality & workflow	Tests, analyze, versioning, no codegen
11	Jargon buster	Plain-language glossary of every term
12	Likely interview questions	Q&A you can rehearse
13	60-second cheat sheet	Facts to memorize the night before

The 30-second pitch

Memorize this. It is the answer to 'So, what is this project?'

Aura (codename RIZAL) is a mobile app for schools that handles two things: attendance and student governance. Students check in to school events with a face scan; student-government officers create events and watch who is present in real time; school IT staff manage user accounts and branding; and a super admin manages all the schools on the platform. The app runs on Android and iOS from a single codebase written in Flutter, and it does not store data itself - it talks over the internet to an existing cloud backend (a FastAPI server) that owns the database and the rules.

Say it like this

"It's a Flutter mobile app, Android and iOS, that is a new client for an existing FastAPI cloud backend. Four roles - student, governance officer, school IT, and super admin - each get their own workspace. The headline feature is face-scan attendance with anti-spoofing and location check-in."

The shape of the system (client-server)

[Your phone]		[The cloud]
Aura Flutter app	--- HTTPS --->	FastAPI backend
(the CLIENT)	request/JSON	(the SERVER)
draws the UI,	<--- JSON -----	owns the database,
holds no DB	response	rules, and truth

Key mindset for the interview: the Flutter app is the CLIENT. It draws screens and sends requests. The backend is the SERVER and is always the source of truth. This app never changes the backend - it only consumes it.

Absolute basics

For total beginners. If you already code, skim - but the vocabulary here is exactly what interviewers expect you to say correctly.

1.1 What is an 'app', and what is frontend vs backend?

An application (app) is a program a user runs. Modern apps are usually split into two halves:

- **Frontend** the part the user sees and touches - the screens, buttons, and animations. The Aura Flutter app IS the frontend.
- **Backend** the part on a server that the user never sees - it stores data in a database, checks passwords, and enforces rules. Aura's backend is a FastAPI (Python) server in the cloud.

They are separate programs that talk to each other over the internet. You can rebuild one without rebuilding the other.

1.2 The client-server model

The 'client' is whoever asks for something; the 'server' is whoever answers. Your phone running Aura is the client. The cloud machine running the backend is the server. The client sends a request; the server sends back a response. This back-and-forth is the foundation of almost every app you use.

1.3 What is an API? A REST API? An endpoint?

- **API** Application Programming Interface - a menu of operations one program offers to another. The backend's API is the list of things the app is allowed to ask for.
- **Endpoint** one item on that menu, written as a URL path, like `/api/events/` or `/token`. Each endpoint does one job.
- **REST** a popular style of API that uses normal web verbs - GET to read, POST to create, PATCH/PUT to update, DELETE to remove.

```
GET    /api/events/      -> read the list of events
POST   /api/events/      -> create a new event
PATCH /api/events/{id} -> edit one event
POST   /api/attendance/manual-> mark a student present
POST   /token         -> log in, get a token
```

1.4 What is JSON?

JSON (JavaScript Object Notation) is the text format the app and backend use to exchange data. It is just labelled values - easy for both humans and machines to read. When the app asks for an event, the server replies with JSON like this:

```
{
  "id": 142,
  "title": "General Assembly",
  "status": "ongoing",
  "location": "Gymnasium",
  "radius_m": 80
}
```

The app's job is to turn this JSON text into a Dart object it can display, and turn Dart objects back into JSON when sending. That two-way translation is called serialization (see Part 6).

1.5 Framework, SDK, library, package

- **Framework** a big, opinionated toolkit that gives your app its overall shape. Flutter is a framework.
- **SDK** Software Development Kit - the tools + libraries you install to build for a platform. The Flutter SDK and the Dart SDK.
- **Library / package** reusable code someone else wrote that you plug in. Aura uses packages like `dio`, `go_router`, and `fl_chart` from `pub.dev` (Dart's package store).

1.6 'Native', 'compiled', 'cross-platform'

- **Native** an app built specifically for a device's own platform (real Android/iOS widgets and speed), not a website in a wrapper.
- **Cross-platform** one codebase that produces apps for several platforms. Flutter is cross-platform: the same Dart code builds the Android APK and the iOS app.
- **Compiled** Dart is compiled to fast native machine code for release builds, which is why Flutter apps feel smooth.

Flutter & Dart fundamentals

The core ideas you must be able to explain about the technology this app is built with.

2.1 What is Flutter?

Flutter is Google's open-source UI toolkit for building apps from a single codebase. Instead of using the operating system's built-in buttons, Flutter draws every pixel itself with its own high-speed rendering engine (Impeller). That is why a Flutter app looks identical on Android and iOS and can animate smoothly. Aura targets Android and iOS, and also runs in a web browser for quick previews.

2.2 What is Dart?

Dart is the programming language Flutter uses. It is modern, typed, and object-oriented. Two Dart features matter a lot here: it supports asynchronous code (talking to the network without freezing the screen) and it compiles to fast native code. Aura uses Dart 3.

2.3 'Everything is a Widget'

In Flutter, every visible thing - and many invisible things - is a Widget: a button, text, padding, a whole screen, even the app itself. You build a screen by nesting widgets inside widgets, forming a widget tree. A small widget like a status chip is composed of a coloured box widget + an icon widget + a text widget.

```
Scaffold                (the page frame)
+- AppBar                (top bar with a title)
+- Column                (stacks children vertically)
  +- Text                ('General Assembly')
  +- StatusChip           (green dot + 'Present')
  +- AuraButton           ('Sign out')
```

2.4 StatelessWidget vs StatefulWidget

- **StatelessWidget** draws something that never changes by itself - given the same inputs it always looks the same (e.g. a logo, a label).
- **StatefulWidget** can change over time and rebuild itself - e.g. a screen whose contents update when data loads or a toggle flips.

Note: in Aura most changing state lives in Riverpod (Part 4) rather than in StatefulWidget, so widgets stay simple and the logic is testable.

2.5 The build() method and rebuilding

Every widget has a build() method that returns the widget tree to display. When the underlying data changes, Flutter calls build() again and efficiently updates only what actually changed on screen. You describe WHAT the UI should look like for the current data; Flutter figures out HOW to update the pixels. This is called a declarative UI.

2.6 Hot reload

Hot reload is Flutter's superpower for developers: change the code and the running app updates in under a second, keeping its current state. It makes tuning layouts and animations fast. (A 'hot restart' is a bit slower and resets state.)

2.7 Material 3

Material Design is Google's design language; Material 3 ('Material You') is its latest version. Flutter ships ready-made Material widgets (buttons, cards, app bars). Aura uses Material 3 and then layers its own design system and brand colours on top (Part 8).

2.8 pubspec.yaml and packages

pubspec.yaml is the app's manifest: its name, version, and its list of dependencies (packages it pulls from pub.dev). Running 'flutter pub get' downloads them. Aura's key packages:

- **flutter_riverpod** state management
- **go_router** screen navigation + route guards
- **dio** HTTP networking to the backend
- **fl_chart** charts and graphs
- **google_fonts** Manrope + JetBrains Mono fonts
- **camera + google_mlkit_face_detection** face capture for attendance
- **geolocator + flutter_map + native_geofence** location and geofencing
- **flutter_secure_storage** safely store the login token

2.9 Asynchronous code: Future, async, await

Talking to the network takes time. If the app waited and froze, the UI would lock up. Instead, network calls return a Future - a promise of a value that will arrive later. The keywords `async` and `await` let you write this naturally: 'await' pauses just that function until the result is ready, while the rest of the app keeps running.

```
Future<List<AppEvent>> ongoing() async {  
  final res = await _client.get('/api/events/ongoing');  
  return parse(res.data); // runs after the reply arrives  
}
```

How this app is structured

Architecture is the #1 thing senior interviewers probe. Learn this section well.

3.1 The big picture: layers

Aura separates code into layers so each piece has one job. Data flows in a predictable direction:

```

SCREEN (widget)           what the user sees
  | watches
PROVIDER (Riverpod)       holds state, decides logic
  | calls
REPOSITORY                knows which endpoint to hit
  | uses
DIO CLIENT (network)      sends HTTP, attaches token
  | <----> JSON
BACKEND (FastAPI)         database + rules (the truth)
  
```

Why bother? Each layer can be tested and changed on its own. If an endpoint moves, only the repository changes - screens are untouched.

3.2 Folder layout

The code lives under `frontend-app/lib`, split into three top folders:

- **lib/core** app-wide foundations: networking (`dio_client`), theme, auth, routing helpers, reusable widgets (buttons, cards, chips).
- **lib/shared** plain data models shared everywhere (Event, Attendance, Governance...) and small utilities (date formatting).
- **lib/features** one folder per feature area. Each is self-contained.

The feature folders are: auth, student, governance, schoolit, admin, events, attendance, gather, liveness, assistant (AI), reports, sanctions, notifications, help, and shell (the navigation frame).

3.3 Feature-first + the data / domain / presentation split

Inside a feature, code is grouped by responsibility, not by file type:

- **data/** repositories - the classes that call the backend and return models. Example: `events_repository.dart`.
- **application/ (domain)** providers and controllers - the state and logic for the feature. Example: `governance_providers.dart`.
- **presentation/** the screens and widgets the user sees. Example: `governance_event_monitor_screen.dart`.

Why feature-first?

When you work on, say, attendance, everything about attendance is in one place. New developers find things fast, features don't leak into each other, and you can delete a whole feature by deleting one folder.

3.4 The four workspaces (roles)

After login, the backend tells the app the user's role. The app maps each role to one of four workspaces, each with its own home screen and tabs:

- **Student** see your schedule, check in to events with a face scan, view your attendance analytics.

- **Governance officer** student-government leaders (SSG / SG / ORG) who create events, monitor live attendance, manage members and sanctions.
- **School IT (campus admin)** manage user accounts by college, branding/logo, programs, and set up the student government.
- **Super admin** manage all schools, subscriptions, accounts, and view system logs.

Role names are normalized first (for example the legacy 'campus-admin' is folded to 'school-it') so old and new backends both work. This logic lives in `lib/core/auth/role.dart`.

3.5 Entry point: how the app boots

The starting sequence:

- **main()** the very first function; it starts the app and wraps it in a Riverpod ProviderScope so state is available everywhere.
- **AuraApp** the root widget (`lib/app/app.dart`). It wires up the theme (light/dark + school branding), the motion preference, localization (English/Filipino), and the router.
- **Router** `go_router` (`lib/app/router.dart`) then decides which screen to show based on the session - see Part 5.

State management with Riverpod

'State' = data that can change while the app runs. Managing it cleanly is what separates a tidy app from a tangled one.

4.1 What problem does it solve?

Many screens need the same changing data: who is logged in, the current theme, the live attendee list. Passing that data by hand through every widget is painful and error-prone. A state-management tool gives any widget a clean way to read shared data and rebuild automatically when it changes. Aura uses Riverpod.

4.2 The core ideas

- **Provider** a named container for a piece of state or a service. Example: `dioClientProvider` provides the network client; `sessionControllerProvider` provides who is logged in.
- **Notifier** a class that holds state and exposes methods to change it. When it updates, every widget watching it rebuilds. Aura uses hand-written Notifiers.
- **`ref.watch(x)`** subscribe to a provider and rebuild when it changes (used inside `build`).
- **`ref.read(x)`** read a provider once without subscribing (used inside event handlers, e.g. on a button tap).

```
// A screen watches a provider and rebuilds when data arrives:
class GovernanceHome extends ConsumerWidget {
  Widget build(context, ref) {
    final events = ref.watch(eventAttendeesProvider);
    return events.when(
      data: (list) => AttendeeList(list),
      loading: () => AuraSkeleton(), // shimmer placeholder
      error: (e, _) => ErrorState(e),
    );
  }
}
```

4.3 'No codegen' - a deliberate choice

Some Riverpod projects auto-generate boilerplate with `build_runner`. Aura deliberately writes its Notifiers and JSON models by hand (no codegen, no freezed). The trade-off: a little more typing, but the code is explicit, easy to read, and there is no generated-file step to break the build. Good thing to mention if asked about project conventions.

4.4 A real example: the session

`sessionControllerProvider` holds the login state: unknown -> authenticated or signed-out, plus flags like 'needs password change' or 'needs face check'. The router watches it; the Dio client tells it to log out on a 401. One source of truth, many readers.

Navigation with go_router

How the app decides which screen to show and protects screens behind login.

5.1 Routes

go_router maps URL-like paths to screens. Aura's main routes are /splash, /login, /change-password, /face-verify, /register-face, and one per workspace: /student, /governance, /workspace (school IT), /admin. Using paths (like the web) makes navigation predictable and testable.

5.2 Redirect guards - the gate

The most important navigation idea here is the redirect guard: before showing any screen, the router runs a function that can send the user somewhere else based on the session. This enforces a strict order so a logged-out user can never land on a private screen:

```
if session unknown OR splash not done -> /splash
else if not logged in                 -> /login
else if must change password          -> /change-password
else if privileged user owes face check -> /face-verify
else if student has no face on file    -> /register-face
else                                   -> your workspace home
```

This pure function (resolveAppRedirect) is easy to unit-test, and the router re-runs it whenever the session changes, so logging out instantly bounces you back to /login.

5.3 Nice transitions: Hero

A Hero animation makes an element (like a card's image) appear to fly and morph into the next screen, giving a polished card-to-detail transition. Aura uses Hero for that effect.

Talking to the backend

How the app actually fetches and sends data - a favourite interview area.

6.1 Dio: the HTTP client

Dio is the package Aura uses to make HTTP requests. Aura wraps it in a thin DioClient (lib/core/network/dio_client.dart) so every request shares the same setup: base URL, timeouts, headers, auth, error handling, and offline cache. Screens never call Dio directly - repositories do.

6.2 REST verbs the client exposes

- **get()** read data (list events, fetch a profile).
- **post()** create something or perform an action (create event, mark present, log in).
- **put()** / **patch()** replace / partially update (edit an event, change settings).
- **delete()** remove something.

6.3 Authentication: JWT bearer tokens

After you log in, the backend returns a JWT (JSON Web Token) - a signed string that proves who you are. The app stores it securely and attaches it to every future request in an 'Authorization: Bearer <token>' header. The server reads the token to know who is asking and what they may do. This is done automatically by a Dio interceptor - a hook that runs on every request and response.

```
// Interceptor (simplified):
onRequest: attach 'Authorization: Bearer <token>'
onResponse: cache successful GETs for offline use
onError:    if status == 401 -> log the user out
            (but NOT for the /token login endpoint -
             a 401 there just means wrong password)
```

6.4 The login flow

Login posts a form to POST /token (grant_type=password, username, password), with /api/token as a fallback. The reply is the JWT plus rich metadata: the user's roles, the school's branding (logo, colours), and feature flags. From then on the bearer token rides along on every call.

6.5 Models and serialization (fromJson / toJson)

The backend speaks JSON; the app prefers strongly-typed Dart objects. Each model class has a fromJson() that builds the object from the JSON map, and toJson() to go the other way. These are written by hand (no codegen). Example models: AppEvent, AttendanceRecord, EventAttendee, GovernanceUnitSummary.

```
factory AppEvent.fromJson(Map<String,dynamic> j) => AppEvent(
  id:      j['id'] as int,
  title:   j['title'] as String,
  status:  j['status'] as String,
);
```

6.6 Pagination

Long lists arrive a page at a time. The backend wraps them in an envelope: { data, page, total, total_pages, limit, next, prev }. Aura's Paginated<T> class unwraps either that envelope OR a plain JSON list, so the rest

of the app handles both shapes the same way. Some screens loop `page=1..total_pages` to load everything and de-duplicate.

6.7 Error handling

Any failed request is converted into a single, friendly `ApiException`. FastAPI's error shapes (`{detail}` or a 422 validation error) are decoded so the UI can show a clear message instead of a raw crash.

6.8 Offline cache

Successful GET responses are cached. If a later GET fails due to a network drop, the client quietly serves the cached copy so the app still shows something useful instead of an error.

Key features

What the app actually does. Be ready to describe two or three of these in detail.

7.1 Face-scan attendance

A student checks in by looking at the camera. The app captures a face (using Google ML Kit face detection) and sends it to the backend, which verifies identity and records the check-in / sign-out. Officers can also mark students present manually (POST /api/attendance/manual) when needed.

7.2 Liveness / anti-spoofing

To stop someone checking in with a photo or video of another student, the app runs an on-device liveness check (the Silent-Face anti-spoofing model) as a fast first filter. The backend stays the final authority. If the on-device plugin fails, it degrades gracefully - the backend-only flow still works. (Jargon: 'spoofing' = faking a real face; 'liveness' = proving the face is a live person.)

7.3 Kiosk 'gather' mode

A device can sit on a counter and scan many faces in a stream for a busy event. The screen is kept awake (wakelock) during the loop, and camera frames are converted (YUV -> RGB -> JPEG) off the camera thread so the UI never stalls.

7.4 Location & geofencing

Events can have a geofence - a circular area (centre + radius) where check-in is allowed. The app reads GPS (geolocator), shows the area on a map (flutter_map), and can use OS-level geofences (native_geofence) so that even when the app is closed, walking into an ongoing event's area fires a check-in notification. Tapping it opens the attendance screen.

7.5 Governance hierarchy (SSG -> SG -> ORG)

Student government is a tree: a campus-wide SSG at the top, college-level Student Governments (SG) under it, and program-level Organizations (ORG) below those. Officers with the right permission can create child units and appoint officers, granting each only the permissions their unit type allows (a backend-enforced whitelist; only SSG can create SG, only SG can create ORG). Permissions like manage_events or manage_attendance gate the buttons the officer sees.

7.6 Live event monitor

During an event, officers see a searchable roster of attendees with status chips that respect the event phase: 'Not yet' (neutral) while the event is running and a student has no record, 'Incomplete' (orange) if they checked in but not out, and 'Absent' (red) only after the event finalizes. The list smart-polls so new check-ins appear without a manual refresh.

7.7 Reports export (PDF / Excel / CSV)

Officers can export an event's attendance to PDF, Excel, or CSV and share it. The heavy file generation runs in a background isolate (compute) so the UI never freezes. (Jargon: an 'isolate' is Dart's way to run work on a separate thread.)

7.8 AI assistant - 'Aura, powered by Jose'

A built-in chat assistant answers questions and can render charts inline (bar/line/pie via fl_chart). It is a separate backend service. A Fast/Think toggle trades speed for depth: Fast uses a slim prompt and no

tools; Think uses the full prompt and tools/charts.

7.9 Help Center, branding, and more

- In-app, role-based Help Center: each user tier sees only the articles relevant to them; searchable, sourced from the project docs.
- School branding: each school's logo and primary/secondary colours are pulled from the login token and applied to the theme live.
- Forgot-password: self-service 6-digit email code flow.
- Google Sign-In, notifications, sanctions, schedule calendars, and student analytics dashboards.

Design & theming

How the app looks consistent and on-brand, and adapts to each school.

8.1 Design tokens

Rather than hard-coding colours and sizes in each widget, Aura reads from design tokens - named values for colour, spacing, and motion kept in one place (`lib/core/theme`). Widgets read `AppTokens.of(context)` and the theme's text styles. Change a token once and the whole app updates consistently.

8.2 Colour, dark mode, and per-school branding

- **Accent** lime #AAFF00 - used sparingly as one highlight per screen; never as large fills or as text on white (too light to read).
- **Dark mode** a true-black (OLED-friendly) dark theme.
- **Status colours** fixed and never branded: green=present, orange=late, red=absent, etc., so status is always unambiguous.
- **School primary** each school's brand colour overrides the accent at runtime; a contrast helper (YIQ) picks readable text on top of it.

8.3 Typography

Manrope is the UI font; JetBrains Mono (a monospace font) is used for numbers, IDs, and timestamps so columns line up. Manrope is a variable font, so the app drives its weight precisely via font variations. Both are bundled as assets (no runtime download).

8.4 Motion and accessibility

- **Motion rules** UI animations stay under 300ms, use ease-out curves (never ease-in), a subtle 0.97 press-scale, and staggered list reveals.
- **Reduced motion** the app honours the OS 'reduce motion' setting (and an in-app toggle) - animations are skipped for users who need that.
- **Touch targets** at least 48dp and spaced apart; every action shows loading and disabled states; skeleton placeholders during loads; haptics only on confirmations.

8.5 Liquid glass navigation (and why it's mobile-only)

An opt-in 'beta' bottom navigation bar uses a frosted-glass blob that slides between tabs - the iOS-26-style 'liquid glass' look. It relies on Flutter's Impeller renderer and shaders, so it works on mobile but falls back to a standard nav bar on the web (the shaders 404 there).

8.6 Responsive shell

The navigation frame adapts to screen width using breakpoints: on a phone (< 600dp) it shows the bottom nav bar; on a tablet or desktop window (>= 600dp) it switches to a vertical sidebar rail. Rotating a tablet keeps you on the same tab because the selected index is shared.

Security

Mobile apps handle credentials and personal data. Interviewers like to hear that you thought about this.

9.1 Token storage and session

The JWT login token is kept in `flutter_secure_storage`, which uses the platform's encrypted store (Android Keystore / iOS Keychain) - not plain shared preferences. If any authenticated request returns 401 (token expired or revoked), the session is driven to 'expired' and the user is bounced to login automatically.

9.2 HTTPS enforcement

Release builds refuse to send requests over plain HTTP to unknown hosts - a defence-in-depth guard at the app layer, on top of Android's `network-security-config` and iOS App Transport Security. A small allow-list covers the known HTTP staging IP and local dev hosts; everything else in production must be HTTPS.

9.3 Certificate pinning hook

There is a wired-but-disabled TLS certificate-pinning hook, ready to switch on once the backend ships HTTPS. Pinning makes the app trust only a specific server certificate, blocking man-in-the-middle attacks.

9.4 Liveness as a security control

The on-device liveness/anti-spoofing check (Part 7.2) is itself a security feature: it raises the bar against photo/video attacks on attendance, while the backend remains the authoritative check.

9.5 Secrets stay out of the code

Real cloud URLs and client IDs are never committed. They are injected at build time with `--dart-define` (or a git-ignored `config/cloud.json`). This keeps secrets out of version control.

Quality & workflow

How the team keeps the app healthy.

10.1 Checks before 'done'

The standard gate before claiming a change works is three clean commands:

```
flutter pub get      # fetch packages
flutter analyze      # static analysis: catch errors/lint issues
flutter test         # run the automated tests (40+ pass)
```

'Static analysis' means the tools read the code without running it to flag mistakes. The test suite includes pure-logic unit tests (e.g. the redirect gate, breakpoints, the help catalogue) and UI/widget tests.

10.2 Versioning and the changelog

The project uses Semantic Versioning (SemVer): MAJOR.MINOR.PATCH. While pre-1.0, each new phase bumps the MINOR number and bug-fixes bump the PATCH; 1.0.0 lands when all four workspaces reach full parity. Every change is recorded in CHANGELOG.md, and pubspec.yaml's version stays in sync as <semver>+<build> (e.g. 1.36.3+2083).

10.3 Conventions worth naming in an interview

- Feature-first folders with data / application / presentation layers.
- Hand-written Notifiers and JSON models - no codegen, no freezed.
- Never hard-code colours; always read from theme tokens.
- This Flutter app is a client only - it never modifies the backend.

Jargon buster

Every term in this reviewer, in plain language. Skim it the morning of the interview.

API	A menu of operations one program offers to another (the backend's contract).
Endpoint	One URL on that menu that does one job, e.g. <code>/api/events/</code> .
REST	API style using web verbs GET/POST/PATCH/DELETE to read and change data.
HTTP / HTTPS	The protocol browsers and apps use to talk to servers; HTTPS is the encrypted version.
JSON	Labelled text format for exchanging data between app and server.
Client / Server	Client asks (the app); server answers and owns the data (the backend).
Frontend / Backend	The part users see vs the server part they don't.
Framework	A large toolkit that shapes your app. Flutter is one.
SDK	Software Development Kit - tools to build for a platform.
Package / Library	Reusable third-party code; Dart's store is pub.dev.
Flutter	Google's UI toolkit; draws its own pixels for Android, iOS, web from one codebase.
Dart	The programming language Flutter uses.
Impeller	Flutter's modern rendering engine (powers smooth/glass effects on mobile).
Widget	Any UI building block in Flutter; screens are trees of widgets.
Widget tree	The nested structure of widgets that forms a screen.
StatelessWidget	A widget whose look never changes on its own.
StatefulWidget	A widget that can change and rebuild over time.
build()	The method that returns the widgets to display for the current data.
Declarative UI	You describe what the UI should look like; the framework updates the pixels.
Hot reload	Apply code changes to the running app in ~1s, keeping state.
Material 3	Google's latest design language; Flutter ships Material widgets.
pubspec.yaml	The app manifest: name, version, and dependency list.

Future	A promise of a value that arrives later (async result).
async / await	Keywords to write asynchronous code that doesn't freeze the UI.
Isolate	Dart's way to run work on a separate thread (used for exports).
State	Data that can change while the app runs.
Riverpod	The state-management library Aura uses.
Provider	A named container for state or a service in Riverpod.
Notifier	A class holding state with methods to change it; updates rebuild watchers.
ref.watch / ref.read	Subscribe-and-rebuild vs read-once for provider values.
Codegen	Auto-generated boilerplate code. Aura avoids it (writes by hand).
go_router	The navigation library mapping paths to screens.
Route	A named destination/screen, e.g. /login or /student.
Redirect guard	A function that can re-route the user (e.g. to /login) before a screen shows.
Hero animation	A shared element that flies/morphs between two screens.
Dio	The HTTP networking package used to call the backend.
Interceptor	A hook that runs on every request/response (attaches token, handles 401).
JWT	JSON Web Token - a signed string proving who you are.
Bearer token	A token sent in the Authorization header to authenticate requests.
401 / 422	HTTP errors: 401 = not authenticated; 422 = invalid request data.
Serialization	Converting objects to/from JSON (fromJson / toJson).
Pagination	Returning a long list one page at a time.
Envelope	A wrapper around list data with paging info (data, page, total...).
Geofence	A circular map area (centre + radius) that triggers an action when entered.
Liveness Anti-spoofing	/ Checking a face is a live person, not a photo/video.
Kiosk mode	Device left running to scan many people (the 'gather' flow).
WakeLock	Keeps the screen awake so a long scan loop isn't interrupted.
SSG / SG / ORG	Student-government tiers: campus-wide / college / program.

Permission	A capability flag (e.g. <code>manage_events</code>) gating what an officer can do.
Design token	A named, reusable design value (colour, spacing, motion).
Theme	The app-wide styling Flutter applies; Aura adds light/dark + branding.
Variable font	One font file with a range of weights, set precisely in code.
Monospace	A font where every character is the same width (good for numbers/IDs).
Reduced motion	An accessibility setting to minimize animations.
Haptics	Small vibrations that confirm an action.
Breakpoint	A screen-width threshold that switches the layout (phone vs tablet).
dp	Density-independent pixel - a size unit that looks consistent across screens.
Static analysis	Checking code for errors without running it (flutter analyze).
SemVer	Semantic Versioning: MAJOR.MINOR.PATCH version numbering.
--dart-define	A build flag to inject secrets/config without committing them.
Cleartext	Unencrypted HTTP traffic; blocked in release except a dev allow-list.
Certificate pinning	Trusting only one specific server certificate to block MITM attacks.
FastAPI	The Python web framework the backend is built with.

Likely interview questions

Short, confident answers you can rehearse out loud.

Q In one sentence, what is this project?

A A cross-platform Flutter mobile app (Android + iOS) for school attendance and student governance that acts as a new client for an existing FastAPI cloud backend, with four role-based workspaces.

Q Why Flutter instead of native Android/iOS?

A One Dart codebase ships both platforms, which is faster to build and keeps the UI consistent. Flutter draws its own pixels (Impeller engine), so we get smooth, custom designs and animations like the liquid-glass nav.

Q How do you manage state?

A With Riverpod. Providers hold shared state and services; hand-written Notifiers expose methods to change it. Widgets ref.watch a provider and rebuild automatically. We avoid codegen so the code stays explicit.

Q How does the app talk to the backend?

A Through a thin DioClient wrapper. Repositories call get/post/patch on it; an interceptor attaches the JWT bearer token to every request, caches GETs for offline, converts errors to a single ApiException, and logs the user out on a 401.

Q How does login and auth work?

A POST /token returns a JWT plus metadata (roles, branding, flags). The token is kept in encrypted secure storage and sent as a Bearer header on every call. A 401 on an authed request expires the session and redirects to login.

Q How is the code organized?

A Feature-first: lib/features/<area> split into data (repositories), application (providers/logic), and presentation (screens). Shared foundations live in lib/core and lib/shared. Each layer has one responsibility.

Q How do you stop people landing on screens they shouldn't?

A go_router runs a redirect guard before each route. A pure function checks the session and forces the right destination: splash -> login -> password/face gates -> the correct workspace. It re-runs whenever the session changes.

Q How do you keep attendance honest?

A Face-scan check-in plus an on-device liveness / anti-spoofing model to reject photos and videos, with the backend as the final authority. Events can also be geofenced, so check-in is only valid inside the event's area.

Q What happens if the network drops?

A Successful GETs are cached; if a later GET fails due to a network error, the client serves the cached copy. Long lists paginate, and heavy work like report export runs in a background isolate so the UI never freezes.

Q What security measures are in place?

A Encrypted token storage, automatic logout on 401, a release-build HTTPS guard with a small dev allow-list, a ready certificate-pinning hook, liveness anti-spoofing, and secrets injected via --dart-define instead of being committed.

Q How do you handle different screen sizes?

A A responsive shell with breakpoints: a bottom nav bar on phones (< 600dp) and a vertical sidebar on tablets/desktop (>= 600dp), keeping the selected tab when the window resizes or the device rotates.

Q How is quality maintained?

A Every change must pass flutter analyze (static analysis) and flutter test (40+ tests) cleanly, follows Semantic Versioning, and is recorded in CHANGELOG.md. UI follows a documented design system with theme tokens.

Q What was a tricky part?

A Examples: making attendance status truthful per event phase (showing 'Not yet' instead of 'Absent' while an event is still running), and getting the liquid-glass nav to degrade gracefully on the web where its shaders aren't available.

60-second cheat sheet

The night-before facts. If you remember only this page, you can still hold a conversation.

The one-liner

Flutter (Dart) mobile app, Android + iOS, for school attendance + student governance; a client for an existing FastAPI backend; four role workspaces.

The stack

- **UI** Flutter + Material 3, custom design system, Manrope + JetBrains Mono.
- **State** Riverpod (hand-written Notifiers, no codegen).
- **Routing** go_router with redirect guards.
- **Network** Dio wrapper + JWT bearer interceptor; JSON models by hand.
- **Storage** flutter_secure_storage (token) + shared_preferences (prefs).
- **Charts** fl_chart. Camera/ML: google_mlkit_face_detection + liveness.
- **Location** geolocator + flutter_map + native_geofence.

The four workspaces

Student - Governance officer (SSG/SG/ORG) - School IT (campus admin) - Super admin.

The data flow (recite it)

Screen watches a Provider -> Provider calls a Repository -> Repository uses the Dio client -> Dio talks JSON to the FastAPI backend (the source of truth).

Three things that sound senior

- **Layering** feature-first with data/application/presentation; each layer testable in isolation.
- **Resilience** offline GET cache, graceful liveness fallback, background isolates for heavy work, reduced-motion support.
- **Security** encrypted token store, 401 auto-logout, HTTPS release guard, pinning hook, anti-spoofing, secrets via --dart-define.

Final tip

You don't need to memorize code. Be able to (1) say the one-liner, (2) draw the client-server + layer diagram, and (3) explain ONE feature end to end (face-scan attendance is the best story). That covers ~80% of what a beginner interview will ask.